学习 • 执行 • 核对

12个实战案例
中文拆解

固定版本源码阅读与复现指导

用途 • 架构 • 代码 • 边界 • 练习

面向会一点 Python 的入门者

先运行基础练习，再阅读进阶案例

v1.0 / 2026.10.12 / 图文自学课程

目录

12个Agent实战案例中文拆解 3
案例01 自定义工具调用（Function Tools） 4
实验卡01 自定义工具调用（Function Tools） 5
案例02 持久会话与记忆（SQLite） 6
实验卡02 持久会话与记忆（SQLite） 7
案例03 本地文档问答（Local RAG） 8
实验卡03 本地文档问答（Local RAG） 9
案例04 混合检索与重排（Hybrid RAG） 10
实验卡04 混合检索与重排（Hybrid RAG） 11
案例05 检索纠错（Corrective RAG） 12
实验卡05 检索纠错（Corrective RAG） 13
案例06 带推理的 Agentic RAG 14
实验卡06 带推理的 Agentic RAG 15
案例07 知识图谱与可核对引用 16
实验卡07 知识图谱与可核对引用 17
案例08 RAG 故障诊断 18
实验卡08 RAG 故障诊断 19
案例09 浏览器 MCP Agent 20
实验卡09 浏览器 MCP Agent 21
案例10 多 MCP 服务协作 22
实验卡10 多 MCP 服务协作 23
案例11 多 Agent 研究团队 24
实验卡11 多 Agent 研究团队 25
案例12 多 Agent 旅行规划 26
实验卡12 多 Agent 旅行规划 27

12个Agent实战案例中文拆解

固定快照：2026-10-10收录版本 | 手册：2026-10-12 v1.0

使用方法

先完成三个基础练习，再按兴趣阅读案例。案例源码阅读附件供核对，不是12个可直接启动的项目。

本手册的源码事实来自固定提交。模型、框架和外部服务可变化，复现前需检查实际账户可用性和依赖兼容性。

如需复现完整上游项目，先克隆并切换固定版本，再进入各案例目录。下列命令仅准备公开源码，不包含模型配置：

```
git clone https://github.com/Shubhamsaboo/awesome-llm-apps.git
cd awesome-llm-apps
git checkout 4bf51ab704fb2c5b3803cd5191b30d7dcdb51dc2
```

每个案例包含用途、架构、关键代码、环境、步骤、核对项和练习答案。上游12例均未在本次制作中运行验收；本包独立3例另有离线测试记录。

案例01 自定义工具调用（Function Tools）

让计算回到确定性的程序

验证状态：已读取固定版本README与实现源码，17个原有实现入口已核对Git blob；未安装、未启动、未调用该上游案例的模型或服务。

用途与学习目标

适合学习把业务函数注册为 Agent 工具。上游计算器包含基本运算、温度换算、复利、百分比、统计和四舍五入工具。它说明模型如何选择函数，并不意味着所有参数都已被充分约束。建议先完成本包 tool_agent.py，再阅读这个框架版本。

架构

流程：用户问题 → Google ADK 的 LlmAgent → 注册的计算函数 → 结构化结果 → 模型答复

关键代码与机制

calculator_agent/agent.py 的 root_agent 使用 LlmAgent，tools 参数传入六个 Python 函数。模型名称为固定源码中的 gemini-3-flash-preview，不应把它当作当前一定可用的型号。

tools.py 的 calculate_basic_math 先过滤字符再调用 eval。字符过滤会改变原始表达，例如一个包含非法字符的输入可能被悄悄改成另一个表达；允许字符内仍能写出重复乘法或幂运算，不能据此认定资源消耗已受控。本课程练习改为 operation、a、b 三项参数和明确四种分支。

calculate_compound_interest 检查本金、年数和复利次数；calculate_statistics 拒绝空列表并计算统计量。学习时重点比较：不同函数对输入的约束是否一致，错误返回字段是否稳定，成功结果能否独立复算。

环境依赖

上游 requirements.txt 要求 google-adk>=1.5.0；模型调用使用 Google 服务，需要自行配置该项目的 Google 密钥。框架版本是范围约束，没有完整固定依赖闭包，不能保证未来安装行为相同。

实验卡01 自定义工具调用（Function Tools）

复现步骤

在该案例目录建立独立环境、安装 requirements.txt，按照上游实际存在的 .env.example 配置 Google 变量，然后执行 adk web，选择 calculator_agent。不要直接 python calculator_agent/agent.py 期待一个完整终端应用。

```
python -m pip install -r requirements.txt
adk web
```

先询问“18乘7”，再试除零。模型和工具执行属于可选的上游复现实验。

结果与故障核对

核对工具返回结果是否为126；除零是否返回错误；输出是否声称执行了未注册工具。还应测试缺参数、超长表达式与类型异常。源码中出现“safely”并不等于安全性已验收。

扩展练习

设计一个不使用 eval 的温度换算函数，明确单位与数值类型；写出0℃和100℃的结果。

参考答案

只接受数值 celsius，输出 celsius*9/5+32；0对应32，100对应212。字符串单位先在输入层解析，不能默默删字符再计算。

固定版本来源

上游README

- agent.py | Git blob：ce412ad747dfb722447fc5e3c9fb8d4f09ac9a9f
- tools.py | Git blob：390efb6cfa2cc8f288515e5e1eb25ea448fd1f21

来源许可：Apache-2.0。作者与版权归上游贡献者；本手册增加中文教学组织、流程解释、源码边界与练习。许可证原文见附录。

案例02 持久会话与记忆（SQLite）

持久会话如何从数据库回到模型

验证状态：已读取固定版本README与实现源码，17个原有实现入口已核对Git blob；未安装、未启动、未调用该上游案例的模型或服务。

用途与学习目标

适合观察框架的会话管理、持久化和重启恢复。这个案例的“记得”依赖稳定的 app_name、user_id、session_id 以及数据库文件，不是模型自动访问磁盘。对应主线第09到12课。

架构

流程：Streamlit 输入 → chat(user_id,message) → 查询或创建会话 → Runner → 数据库事件 → 显示答复

关键代码与机制

agent.py 创建 DatabaseSessionService，db_url 指向 sqlite:///sessions.db。Runner 绑定 agent 与该服务。chat 函数根据 user_id 构造 session_id，先 get_session，再按需 create_session，最后运行 Agent 并收集最终响应。

app.py 用 Streamlit 的 session_state 保存当前页面显示的 messages，并调用 chat("demo_user", prompt)。页面消息列表与 SQLite 中的会话不是同一存储层：重新加载页面可能丢失显示列表，但数据库会话仍可能存在。清空页面按钮主要清理页面 state，不能直接推断数据库历史已删除。

固定的 demo_user 适合单人演示，不是多用户认证。源码还把用户 ID 与会话 ID 关联起来，学习时要检查恢复是否使用同一组标识。不要把“新建任意ID”当成“恢复旧会话”。

环境依赖

google-adk>=1.9.0、Streamlit、python-dotenv 和 Google 模型服务；SQLite 在本机写文件。源码中的模型是 gemini-3-flash-preview，实际可用性需自行核对。

实验卡02 持久会话与记忆（SQLite）

复现步骤

在案例目录安装 requirements.txt，按上游环境模板配置密钥，然后启动网页：

```
python -m pip install -r requirements.txt
streamlit run app.py
```

输入一个虚构偏好，停止应用，再在同目录重启并继续同一 demo_user。观察原会话是否被取回；不要删除 sessions.db 后再期待恢复。

结果与故障核对

分别记录页面历史和服务端会话是否恢复。更换工作目录可能创建另一个同名数据库。没有登录功能时，不应把固定 demo_user 的演示部署给多个真实用户。

扩展练习

解释“清空页面但下次模型仍提到旧信息”的可能原因，并提出一项验证方法。

参考答案

页面列表已清空，SQLite 会话可能仍在。检查服务端 session_id 与数据库事件，确认清空动作是否同时删除或重置持久会话。

固定版本来源

上游README

- agent.py | Git blob：cfc93c3021cf537a3ecac480274102d430118003
- app.py | Git blob：6eb5e83afd8101551e8d455b7a53d50b0e49ded3

来源许可：Apache-2.0。作者与版权归上游贡献者；本手册增加中文教学组织、流程解释、源码边界与练习。许可证原文见附录。

案例03 本地文档问答（Local RAG）

本地文档、嵌入、向量库与本地模型

验证状态：已读取固定版本README与实现源码，17个原有实现入口已核对Git blob；未安装、未启动、未调用该上游案例的模型或服务。

用途与学习目标

适合理解一个使用 Ollama、Qdrant 与 Agno 的文档问答应用。它与本包关键词练习不同：上游使用向量库与本地推理服务，首次安装和模型下载仍需要网络。

架构

流程：PDF资料 → Knowledge → OllamaEmbedder → Qdrant → 检索上下文 → Ollama模型 → AgentOS

关键代码与机制

local_rag_agent.py 构造 Qdrant，地址是 localhost:6333，集合名 thai-recipe-index。Knowledge 接收这个向量库，然后 add_content 下载一个公开食谱 PDF。脚本没有内置你自己的资料目录遍历，需要修改资料导入位置。

生成模型使用 Ollama(id="llama3.2")，最终通过 AgentOS 暴露应用。agent_os.serve 在脚本入口启动服务。这里的“本地”指模型和向量库可以在本机运行，不应忽略默认资料仍是远程 URL。

README 建议拉取 openhermes 用于嵌入，但源码实例 OllamaEmbedder() 没有显式填模型名。复现时应检查所安装 Agno 版本的默认嵌入型号并显式设定，而不是假定拉取任意模型就够用。生成模型与嵌入模型的职责不同。

环境依赖

requirements.txt 中有 agno>=2.2.10、qdrant-client、Ollama 相关依赖；还需安装运行 Ollama 和 Qdrant。硬件内存与磁盘需求取决于实际模型，课程未测量该上游案例的占用。

实验卡03 本地文档问答（Local RAG）

复现步骤

先启动本机 Qdrant，再拉取生成与实际使用的嵌入模型。Docker 示例仅把 Qdrant 暴露在回环地址：

```
docker run --name course-qdrant -p 127.0.0.1:6333:6333 qdrant/qdrant
ollama pull llama3.2
ollama pull openhermes
python -m pip install -r requirements.txt
python local_rag_agent.py
```

确认 Ollama 服务正在运行；上述嵌入模型名须与源码配置一致。应用访问地址以实际控制台输出为准。

结果与故障核对

检查首次导入是否重复执行、集合是否创建、资料是否真被读取。对有答案和无答案问题分别测试；不以网页启动代表检索正确。远程PDF不可访问时先替换为可访问教学资料。

扩展练习

把资料源换成一份自己的公开文本，说明生成模型和嵌入模型各负责什么。

参考答案

嵌入模型负责文档与问题表示，生成模型根据检索上下文生成文本。更换嵌入模型需重建兼容索引；只更换生成模型通常不要求重算文档向量，但仍须重新验证答案。

固定版本来源

上游README

• local_rag_agent.py | Git blob：6d3473095f12262bd5c1f2e751e6b70b96de6035

来源许可：Apache-2.0。作者与版权归上游贡献者；本手册增加中文教学组织、流程解释、源码边界与练习。许可证原文见附录。

案例04 混合检索与重排（Hybrid RAG）

混合检索后重排，再决定回答依据

验证状态：已读取固定版本README与实现源码，17个原有实现入口已核对Git blob；未安装、未启动、未调用该上游案例的模型或服务。

用途与学习目标

适合观察多路召回、重排和生成之间的接口。上游使用 RAGLite、Streamlit、嵌入服务、Cohere 重排与 Anthropic 生成，配置比基础练习多。

架构

流程：上传PDF → insert_document → hybrid_search → retrieve_chunks → rerank_chunks → rag生成

关键代码与机制

main.py 的 initialize_config 构建 RAGLiteConfig。固定实现写有嵌入模型 text-embedding-3-large 和 Cohere reranker，模型与服务名称属于该快照。perform_search 先 hybrid_search 取10个候选，再 retrieve_chunks，再 rerank_chunks。

主界面保存配置、导入文档，并在查询时调用 perform_search。若没有候选，handle_fallback 直接调用通用模型回答。这个回退会改变回答依据：用户若以为答案始终来自上传资料，就容易把常识答复当成有来源的结论。

数据库默认可使用本地 SQLite，README 同时介绍外部数据库选项。先用最小本地路径排除数据库连通问题，再检查嵌入、重排与生成三个服务。任何一层失败都应单独显示，不要统一写成模型错误。

环境依赖

固定 requirements 包含 raglite==0.2.1、Streamlit、rerankers、Cohere 与相关模型客户端。需要嵌入、生成与重排三类账户配置。上游依赖存在固定与范围混用，实际安装仍需验证。

实验卡04 混合检索与重排（Hybrid RAG）

复现步骤

安装固定目录的 requirements，并按 README 安装其指定的 spaCy 模型包。该外部包应自行检查官方来源后安装。启动：

```
python -m pip install -r requirements.txt
streamlit run main.py
```

在侧栏配置三类密钥与数据库路径，上传公开PDF，再先问一个明确写在文档中的事实。

结果与故障核对

保存候选与重排结果，观察正确片段是否进入最终上下文。另问一个文档没有答案的问题，确认回退答复是否被明确标注。课程未测量重排带来的效果或延迟。

扩展练习

为这个案例设计“仅依据文档”开关，说明无候选时应怎样返回。

参考答案

开启时，无候选或证据不足就返回说明，不调用通用知识回退；关闭时允许回退，但答案必须标明资料外信息，并与文档结论区分。

固定版本来源

上游README

• main.py | Git blob：07af8a0cc47ce06e038ff99626c53ae7bd946620

来源许可：Apache-2.0。作者与版权归上游贡献者；本手册增加中文教学组织、流程解释、源码边界与练习。许可证原文见附录。

案例05 检索纠错（Corrective RAG）

检索失败时，改写的是查询而不是目标

验证状态：已读取固定版本README与实现源码，17个原有实现入口已核对Git blob；未安装、未启动、未调用该上游案例的模型或服务。

用途与学习目标

适合阅读 LangGraph 状态图。上游实现把检索、相关性判别、查询改写、外部搜索和生成分成节点，比只加一句“请纠错”更容易观察分支。

架构

流程：retrieve → grade_documents → 足够则generate；不足则transform_query → web_search → generate

关键代码与机制

corrective_rag.py 定义状态并建立 StateGraph，retrieve 从向量检索器取文档，grade_documents 用模型判断片段相关性。decide_to_generate 根据状态选择生成或改写。transform_query 将问题改写，再进入 web_search。

固定图里的 web_search 直接连向 generate，而不是再次回到相关性评分节点。因此它不是无限循环纠错，也没有展示“搜索后再评分”的完整闭环。分析源码时应以真实图边为准，不能只照标题想象机制。

相关性输出需要解析 JSON 的 score 字段。格式异常、打分误判和外部搜索失败都会影响结果。源码使用 Qdrant、嵌入服务、Anthropic 和 Tavily；缺少搜索配置时应检查返回分支是否仍然沿用弱证据。

环境依赖

LangChain、LangGraph、Qdrant、Streamlit 与多类API。requirements 固定部分版本，例如 LangChain 0.3.12；安装现代版本可能改变接口。数据源支持情况由 loader 分支决定，不应把所有文件都当可读。

实验卡05 检索纠错（Corrective RAG）

复现步骤

先启动 Qdrant，安装案例 requirements，然后启动：

```
python -m pip install -r requirements.txt
streamlit run corrective_rag.py
```

配置生成、嵌入、Qdrant及搜索服务，选择公开资料。先测试一题能够直接检索回答，再测试一题需要改写，并记录每个节点输出。

结果与故障核对

核对 original question 是否被保留；搜索来源是否可靠；改写是否丢失约束；没有证据时是否仍生成确定答案。重点测试搜索不可用和评分JSON异常。

扩展练习

把一次外部搜索后的资料重新评分，画出新增分支与停止规则，不需要真正调用API。

参考答案

可在 web_search 后增加 grade_external，相关则生成，不相关则返回证据不足。设置最多一次改写和一次搜索，避免循环；记录原问题与每次查询。

固定版本来源

上游README

• corrective_rag.py | Git blob：853a74a5a21bf2731668e11effb2abeef764f1c3

来源许可：Apache-2.0。作者与版权归上游贡献者；本手册增加中文教学组织、流程解释、源码边界与练习。许可证原文见附录。

案例06 带推理的 Agentic RAG

推理工具与知识检索是两项能力

验证状态：已读取固定版本README与实现源码，17个原有实现入口已核对Git blob；未安装、未启动、未调用该上游案例的模型或服务。

用途与学习目标

适合观察 Agent 在知识库上组织回答。上游将 Agno、LanceDB、OpenAI 嵌入、Gemini 生成与 ReasoningTools 组合，教学重点是信息流和证据检查。

架构

流程：URL资料 → Knowledge/LanceDB → 向量检索 → Agent与推理工具 → 流式答复与引用

关键代码与机制

rag_reasoning_agent.py 的 load_knowledge 建立 LanceDb，search_type 为 vector，嵌入器为 OpenAIEmbedder。load_agent 将该知识库与 Gemini 模型、ReasoningTools 组合。它不是关键词检索与向量融合的演示。

页面 state 保存 knowledge_urls 与 urls_loaded，避免同一页面状态反复导入。加载资料使用远程URL，新增URL后立即导入；若URL内容发生变化，仍要考虑索引如何更新，不能仅靠“已加载”集合判断版本。

流式界面收集答案与 citations；引用字段是否出现，取决于框架和模型响应。推理工具帮助组织过程，不会自动证明最终结论。阅读时把“有推理展示”与“证据支持了回答”分别检查。

环境依赖

Agno、Streamlit、LanceDB、Google模型与嵌入服务。需要 Google 和嵌入服务密钥。资料抓取与模型请求均可能联网，不能把此案例称为完全本地。

实验卡06 带推理的 Agentic RAG

复现步骤

在案例目录安装 requirements，启动：

```
python -m pip install -r requirements.txt
streamlit run rag_reasoning_agent.py
```

填入两类密钥，使用公开URL，确认导入成功后提问。所有模型名与数据库接口都需按实际依赖版本复核。

结果与故障核对

记录检索来源、最终断言与引用。测试一个跨段落问题和一个资料外问题。不要把模型展示的推理文本直接视作真实内部过程或正确性证明。

扩展练习

给一次回答分别评“是否有引用”“引用是否支持结论”“是否遗漏例外”三项。

参考答案

三项独立记录，不能用一项替代其余两项。例如有URL但原文没有相关条件，引用存在通过，支持性不通过。

固定版本来源

上游README

• rag_reasoning_agent.py | Git blob：336c7a4afce1131f82d443b0cf963d8ed2593eb6

来源许可：Apache-2.0。作者与版权归上游贡献者；本手册增加中文教学组织、流程解释、源码边界与练习。许可证原文见附录。

案例07 知识图谱与可核对引用

知识图谱引用：找到编号还不够

验证状态：已读取固定版本README与实现源码，17个原有实现入口已核对Git blob；未安装、未启动、未调用该上游案例的模型或服务。

用途与学习目标

适合理解实体、关系、多跳扩展和来源追踪。上游使用 Ollama 与 Neo4j，构建能展示来源的图谱问答；源码存在几个很适合练习核查的边界。

架构

流程：文本 → 实体关系抽取 → Neo4j → 初步文本匹配 → 多跳扩展 → 答复编号 → 来源映射

关键代码与机制

KnowledgeGraphManager 写入实体和关系，保存 source_doc、source_chunk 等信息。find_related_entities 做限定跳数的图遍历。semantic_search 虽然这样命名，实际使用词项与 CONTAINS 文本匹配，没有文档嵌入向量；不要仅凭函数名判断算法。

generate_answer_with_citations 根据初始匹配结果扩展上下文，要求模型用编号引用。随后用正则提取编号，在 source_map 中找对应项。该检查能确认编号属于已提供来源，不能确认每个具体断言都得到原文支持。

Citation 的 confidence 被固定写成0.85，不是实测概率。clear_graph 使用删除全部节点关系的语句，复现实验必须使用专门的教学数据库，不能连接已有业务图谱。图谱抽取阶段也可能把模型误判变成持久事实，需要核对。

环境依赖

Streamlit、neo4j Python客户端、Ollama；本机需要运行 Neo4j 与 Ollama，并准备生成模型。数据源为教学文本，实际图结构与抽取结果需检查。

实验卡07 知识图谱与可核对引用

复现步骤

先准备独立教学 Neo4j 数据库，启动 Ollama并拉取实际模型，再安装依赖并运行：

```
ollama pull llama3.2
python -m pip install -r requirements.txt
streamlit run knowledge_graph_rag.py
```

配置数据库地址与凭据，在页面添加虚构文本。避免在含真实数据的数据库上使用清空功能。

结果与故障核对

抽查实体是否来自原文；关系方向是否正确；多跳链条是否回答问题；编号是否对应原文。不要把固定0.85写成“85%准确率”或把语句相关当成事实支持。

扩展练习

用“甲公司收购乙公司，乙公司位于杭州”构造两跳问题，并指出模型抽取可能出错的地方。

参考答案

问题可为“甲公司收购的公司在哪”。需要甲→收购→乙→位于→杭州两项事实。实体混淆、关系方向反转或原文并未说明地点都可能使回答失效。

固定版本来源

上游README

- knowledge_graph_rag.py | Git blob：3db32e406cf93284712b4a219d3099558869656f

来源许可：Apache-2.0。作者与版权归上游贡献者；本手册增加中文教学组织、流程解释、源码边界与练习。许可证原文见附录。

案例08 RAG 故障诊断

故障分析助手提供建议，不代替工程定位

验证状态：已读取固定版本README与实现源码，17个原有实现入口已核对Git blob；未安装、未启动、未调用该上游案例的模型或服务。

用途与学习目标

适合学习把故障描述映射到可复用模式。案例是一个框架依赖较少的终端程序，通过模型输出诊断、证据需求和处理建议。

架构

流程：选择或输入故障 → 故障模式提示 → 模型请求 → 诊断建议 → 人工验证

关键代码与机制

rag_failure_diagnostics_clinic.py 包含12类模式：证据漂移、切分边界、嵌入不匹配、索引陈旧、查询路由、长链漂移、工具误用、记忆问题、评测盲区、启动顺序、配置差异和相互干扰。

build_system_prompt 组织模式说明，choose_bug_description 提供示例或接受自定义输入，make_client_and_model 从环境变量读取配置。run_once 调用模型并展示输出。程序没有读取你的实际向量索引或自动修复代码，因此输出是诊断假设。

故障描述越具体，越容易形成可验证建议。写清问题、预期、实际候选、文档版本和最近修改；如果只输入“RAG不准”，模型只能给宽泛的可能性。先定位证据缺失还是生成偏离，再选择改动。

环境依赖

requirements.txt 提供模型客户端依赖。源码默认配置读取 OPENAI_API_KEY、OPENAI_BASE_URL 和 OPENAI_MODEL。兼容端点可以调整，但并非所有服务参数都等价；课程没有验证替换端点。

实验卡08 RAG 故障诊断

复现步骤

安装该目录依赖，按源码配置变量，然后运行：

```
python -m pip install -r requirements.txt
python rag_failure_diagnostics_clinic.py
```

先选上游内置虚构故障，再输入自己的脱敏样本。这个命令会请求模型，不属于本包默认离线练习。

结果与故障核对

输出应把确认事实与假设区分。每条建议配一个能取得的证据，例如检查索引版本或打印候选。没有证据支持时不能把某一模式写成确定根因。

扩展练习

输入“更新政策后还回答旧截止日”，给出两个可能原因和各自检查方法。

参考答案

索引未更新：比较源文件哈希与索引版本；生成忽略新资料：检查实际传入上下文是否已是新截止日。先按数据流确认哪个阶段最早偏离。

固定版本来源

上游README

• rag_failure_diagnostics_clinic.py | Git blob：9e315f8523cfa5b44a0a141801556d8a662b5197

来源许可：Apache-2.0。作者与版权归上游贡献者；本手册增加中文教学组织、流程解释、源码边界与练习。许可证原文见附录。

案例09 浏览器 MCP Agent

浏览器 MCP 的服务连接与页面结果

验证状态：已读取固定版本README与实现源码，17个原有实现入口已核对Git blob；未安装、未启动、未调用该上游案例的模型或服务。

用途与学习目标

适合观察 Host 如何经 MCP 获得 Playwright 工具。上游使用 Streamlit 与 mcp-agent，额外依赖 Node.js、浏览器工具服务和模型配置。

架构

流程：网页输入 → MCPApp → browser Agent → playwright服务 → 模型调用工具 → 页面操作与答复

关键代码与机制

main.py 的 setup_agent 创建 MCPApp 的异步上下文，构造 server_names=["playwright"] 的 Agent，初始化后 attach_llm，再 list_tools。run_mcp_agent 负责检查模型配置并生成文本。

独立配置 mcp_agent.config.yaml 将 playwright 指向 npx，参数是 @playwright/mcp@latest。仓库固定提交并没有固定这个外部服务版本，因此长期复现时需要额外记录实际安装版本。配置中的模型默认来自旧快照，实际可用性需核对。

该应用手动进入异步上下文并将对象存在 session_state。学习时要检查页面重载、异常与退出时资源能否释放。工具连接成功和页面任务完成是两项不同信号；不能只看 initialized 为真。

环境依赖

Streamlit、mcp-agent>=0.0.14、Node.js/npm、Playwright MCP 和模型服务。实际浏览器组件可能需要下载。配置文件与源码共同决定服务名称、模型和连接方式。

实验卡09 浏览器 MCP Agent

复现步骤

先检查 Node.js 与 npm，安装 requirements，按同目录 mcp_agent.config.yaml 和 secrets 示例配置模型，然后执行：

```
node --version
npm --version
python -m pip install -r requirements.txt
streamlit run main.py
```

用公开文档页的只读任务做第一项实验。首次启动外部服务会下载组件，应自行检查来源与版本。

结果与故障核对

核对工具列表与服务名一致；观察页面是否实际到达目标；点击后重新读取状态；记录可见成功信号。页面提示不等于用户授权，不执行无关上传或写入。

扩展练习

设计“从公开文档找安装命令”的验收项，至少写出一个页面异常和处理方式。

参考答案

验收为找到对应版本的命令与原页面链接。弹窗遮挡时先观察并处理正常弹窗；访问受限时说明限制，不绕过登录或访问控制。

固定版本来源

上游README

• main.py | Git blob：8d83dbf1065ddd983107bfda9082d8e2b3aa57b3

来源许可：Apache-2.0。作者与版权归上游贡献者；本手册增加中文教学组织、流程解释、源码边界与练习。许可证原文见附录。

案例10 多 MCP 服务协作

多 MCP 服务协作的权限与状态

验证状态：已读取固定版本README与实现源码，17个原有实现入口已核对Git blob；未安装、未启动、未调用该上游案例的模型或服务。

用途与学习目标

适合阅读 Agno 对多个 MCP 服务的组织方式。案例包含仓库、搜索、日历和邮箱服务，但接通服务不代表所有写操作都已被授权。

架构

流程：环境配置 → 多MCP服务上下文 → 单个Agent使用多工具 → 会话数据库 → 交互终端

关键代码与机制

multi_mcp_agent.py 验证三类变量，再随机生成 user_id 与 session_id。它通过 MultiMCPTools 连接四条 npx 服务命令，为单个 Agent 提供工具集合；README 使用“Multi-Agent”描述，实际主文件展示的是一个 Agent 加多个服务。

SqliteDb 指向 tmp/multi_mcp_agent.db。每次启动会话ID随机产生，旧数据库存在不等于自动恢复同一会话。acli_app 接受本次 user_id 和 session_id。按持久化章节的方法检查这些标识如何复用。

env={**os.environ,...} 会把整个进程环境交给子服务；独立实现可改为只传服务必要变量。服务命令也没有统一固定版本，运行时权限与包版本要另外核对。源码里的能力描述不能代替授权和成功结果。

环境依赖

Agno、mcp、python-dotenv、模型客户端、Node.js与各外部服务。GitHub与搜索凭据、日历/邮箱授权各有独立要求，不能将一项密钥当作全部服务都可访问。

实验卡10 多 MCP 服务协作

复现步骤

安装依赖并检查Node工具；按同目录环境模板配置，只在隔离学习账户里尝试：

```
python -m pip install -r requirements.txt
node --version
python multi_mcp_agent.py
```

第一次只做读取与生成草稿任务，先观察哪些子服务真正启动。未完成必要授权时，不声称对应服务可用。

结果与故障核对

检查服务启动失败的行为、继承变量范围、会话恢复和输出凭据风险。测试只读请求是否触发写工具；跨服务写入需要可核对结果ID与去重策略。

扩展练习

把“读取仓库问题并安排复查”拆成读取、草稿和写入三步，并列出必要权限。

参考答案

读取阶段只需要仓库只读；草稿阶段在本机生成标题与日期；写入阶段才需要日历创建权限，必须校验时间与授权并保存事件ID。

固定版本来源

上游README

- multi_mcp_agent.py | Git blob：dc3689541645dd2bf768dc84937c0768edfe17e6

来源许可：Apache-2.0。作者与版权归上游贡献者；本手册增加中文教学组织、流程解释、源码边界与练习。许可证原文见附录。

案例11 多 Agent 研究团队

研究团队：找、读、整合的职责

验证状态：已读取固定版本README与实现源码，17个原有实现入口已核对Git blob；未安装、未启动、未调用该上游案例的模型或服务。

用途与学习目标

适合阅读多 Agent 在信息研究任务中的分工。上游有模型服务版本与 Ollama 版本，都结合 HackerNews、网页搜索和文章读取工具。

架构

流程：研究问题 → HackerNews研究员 + 网页搜索员 + 文章阅读员 → Team协调 → 汇总与来源

关键代码与机制

research_agent.py 构造三个成员：hn_researcher 使用 HackerNewsTools，web_searcher 使用 DuckDuckGoTools，article_reader 使用 Newspaper4kTools；Team 接收成员并生成结果。固定源码实际使用 gpt-4o-mini，README的简写描述不应覆盖源码。

research_agent_llama3.py 改用 Ollama(id="llama3.2")，但网页检索和文章读取仍需要网络，因此“本地模型”不等于整个任务离线。两个文件可帮助你把模型服务与外部工具分开分析。

团队角色的提示表达了希望发生的分工，不保证每次严格按相同顺序执行。结果要保留来源、日期和支持的事实；文章读取失败时不能让整合角色自行补造文章内容。

环境依赖

Streamlit、agno>=2.2.10、HackerNews/搜索/文章读取相关包。模型服务版本需要账户配置；Ollama版本需要本地模型。网络可访问性影响各工具行为。

实验卡11 多 Agent 研究团队

复现步骤

在目录安装 requirements 后，选择一种模型路径：

```
python -m pip install -r requirements.txt
streamlit run research_agent.py
```

或先运行 Ollama并拉取模型，再执行：

```
ollama pull llama3.2
streamlit run research_agent_llama3.py
```

用公开且范围清楚的研究题测试，核对来源和日期。

结果与故障核对

比较单个搜索Agent与团队的来源覆盖、重复项、耗时和结果支持。不能只凭内容更长推断质量更高。案例未在本次制作中测得准确率或收益。

扩展练习

定义一张成员交付表，要求找资料者、读文章者、整合者分别交付哪些字段。

参考答案

找资料者交付URL、标题和时间；阅读者交付原文依据与提取事实；整合者只使用确认事实，并列出冲突与未读取来源。

固定版本来源

上游README

- research_agent.py | Git blob：a0ae20c2e4a000686af3e1ba267cf1dbe7abfe54
- research_agent_llama3.py | Git blob：8bfe1f530472eda8a1cc7ba746dbef2e8bf7c49b

来源许可：Apache-2.0。作者与版权归上游贡献者；本手册增加中文教学组织、流程解释、源码边界与练习。许可证原文见附录。

案例12 多 Agent 旅行规划

旅行规划：协调器不能替代约束检查

验证状态：已读取固定版本README与实现源码，17个原有实现入口已核对Git blob；未安装、未启动、未调用该上游案例的模型或服务。

用途与学习目标

适合观察一个前后端完整度更高的多 Agent 应用。它涉及航班、住宿、餐饮、预算、目的地和行程六类成员，建议作为进阶阅读，不作为首个运行项目。

架构

流程：Next.js表单 → FastAPI → 协调Team → 六类成员 → 行程与预算草稿 → 约束核对

关键代码与机制

backend/agents/team.py 创建 mode="coordinate" 的 Team，members 包含目的地、酒店、餐饮、预算、航班与行程成员。itinerary.py 定义行程 Agent；文件 travel_planning_team.py 只是航班库调用相关片段，不能把它视为全应用启动入口。

进一步核对 backend/main.py 后，确认它从 api.app 导入应用。backend/config/llm.py 的实际模型使用 OpenRouter，包含 Google与OpenAI路由名称；README概述为Gemini，配置细节应以代码为准。主脚本默认监听0.0.0.0，学习启动改用明确的127.0.0.1命令。

根案例目录没有 requirements.txt，后端依赖在 pyproject.toml 与 uv.lock；前端实际目录为 client，拥有 package.json、Prisma与环境示例。不要套用其他案例的 pip install -r requirements.txt 或把前端路径猜成 frontend。

旅行草稿中的营业时间、交通时间、价格与预算约束需另外验证。多个Agent都推荐了同一地点，不代表信息独立可靠，也不意味着完成了真实预订。

环境依赖

后端要求Python>=3.12，使用Agno、FastAPI、数据库与搜索/抓取组件；pyproject声明Agno>=2.3.24。client使用Next.js、React与Prisma。环境模板包括数据库、OpenRouter、搜索、抓取和存储相关变量，按启用路径配置。

实验卡12 多 Agent 旅行规划

复现步骤

先在 backend 按固定 uv.lock 建立环境并配置 .env，然后仅绑定本机启动API：

```
uv sync --frozen
uv run uvicorn api.app:app --host 127.0.0.1 --port 8000
```

再进入 client，按README安装包、配置 .env.local，并针对新建教学数据库初始化Prisma，最后启动网页开发服务。不得对已有业务库运行初始化命令。

```
pnpm install
pnpm prisma generate
pnpm prisma db push
pnpm dev
```

以上是依据源码整理的复现路线，未运行，服务间地址与凭据仍需逐项核对。

结果与故障核对

先验收API健康与表单数据流，再测试日期、每日时间、费用合计、交通可行性与来源时效。预订结果必须有对应交易证据；行程文本不能冒充已预订。

扩展练习

为两日旅行草稿增加确定性检查：日期是否连续、同一天是否冲突、总费用是否超预算。

参考答案

日期按明确时区排序；用时间区间检查重叠并保留交通缓冲；分项价格统一货币再求和与预算比较。缺失价格写明未知，不按零计算。

固定版本来源

上游README

- team.py | Git blob：37326dc268b786585f48e227e94cd6f7638ddb07
- itinerary.py | Git blob：5d2b5075fc675af907f98801be63d38d00d63af2
- travel_planning_team.py | Git blob：d02a95ec0ef67ef1ce007f1845bb45db4bbe6753

来源许可：Apache-2.0。作者与版权归上游贡献者；本手册增加中文教学组织、流程解释、源码边界与练习。许可证原文见附录。